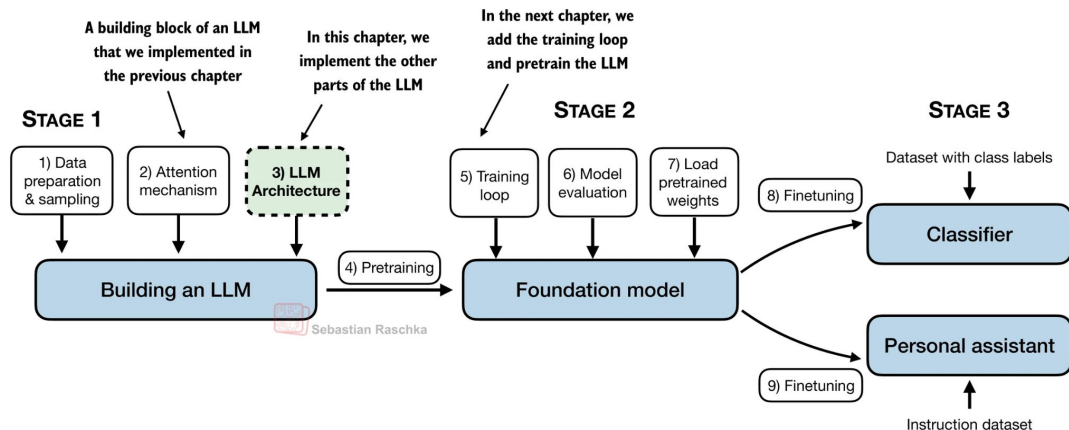


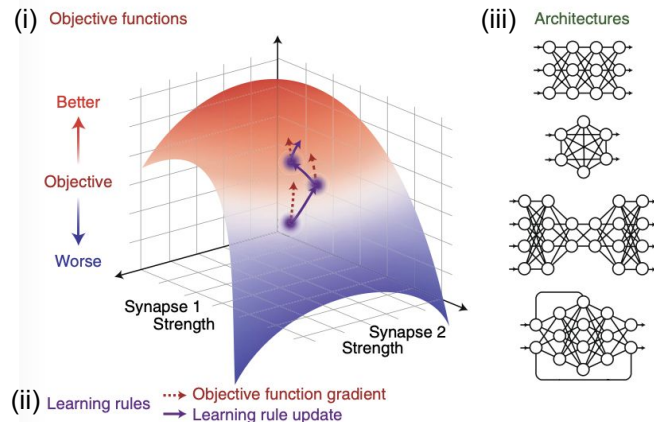
Ch. 4: LLMs From Scratch

Guangyao Qi and Kashi Tuteja

Goal of the Architecture



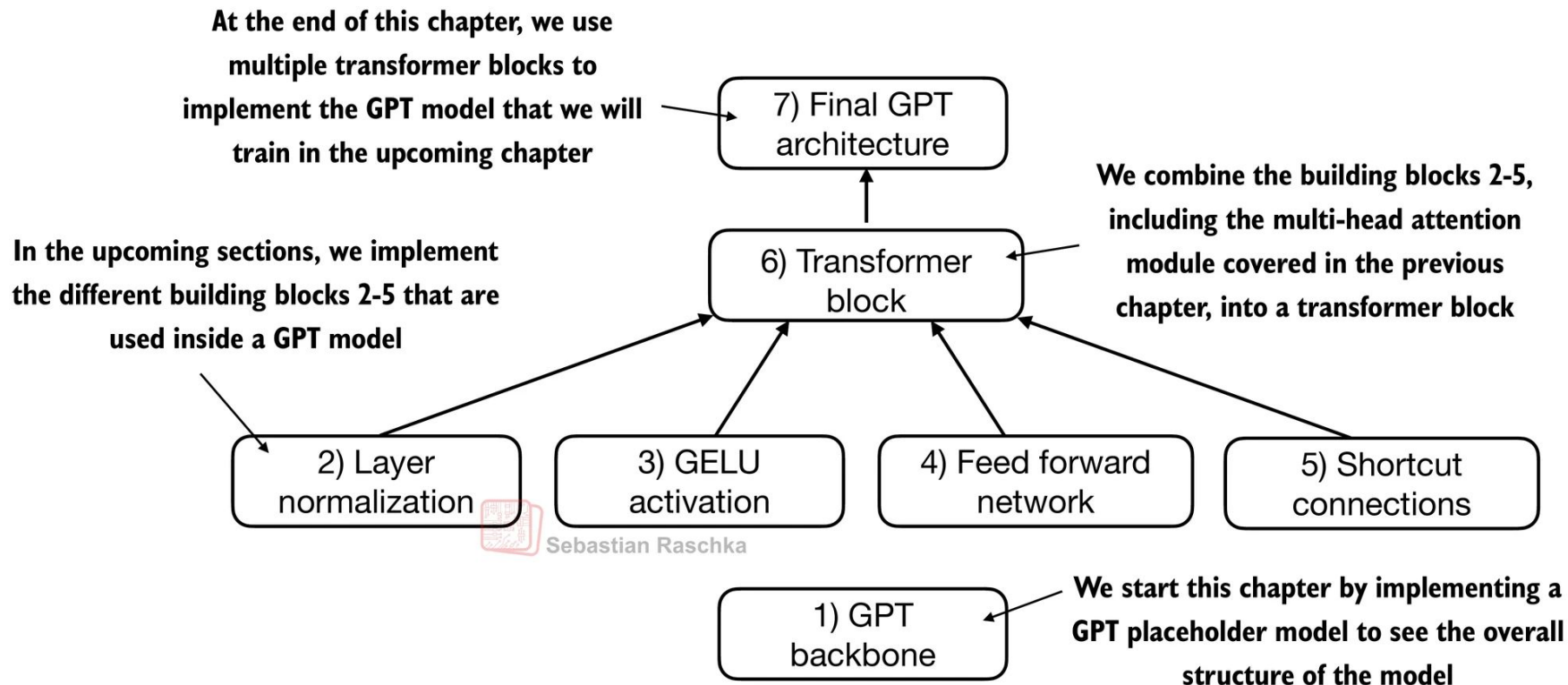
Three core components of ANN design



(Richards et al, 2019)

- (i) Objective functions: the goals of the learning system.
- (ii) Learning rules: how the parameters in a model are updated.
- **(iii) Architectures: how the units are arranged and what operations they can perform in LLMs.** Architectures specify the arrangement of units in the network and determine the flow of information, as well as the computations that are or are not possible for the network to learn.
 - E.g., a CNN (Convolutional Neural Network) is excellent at detecting spatial patterns (edges, shapes) because its architecture enforces local connectivity.
 - But a simple single-layer perceptron (a network with no hidden layers) *cannot* learn non-linear relationships.

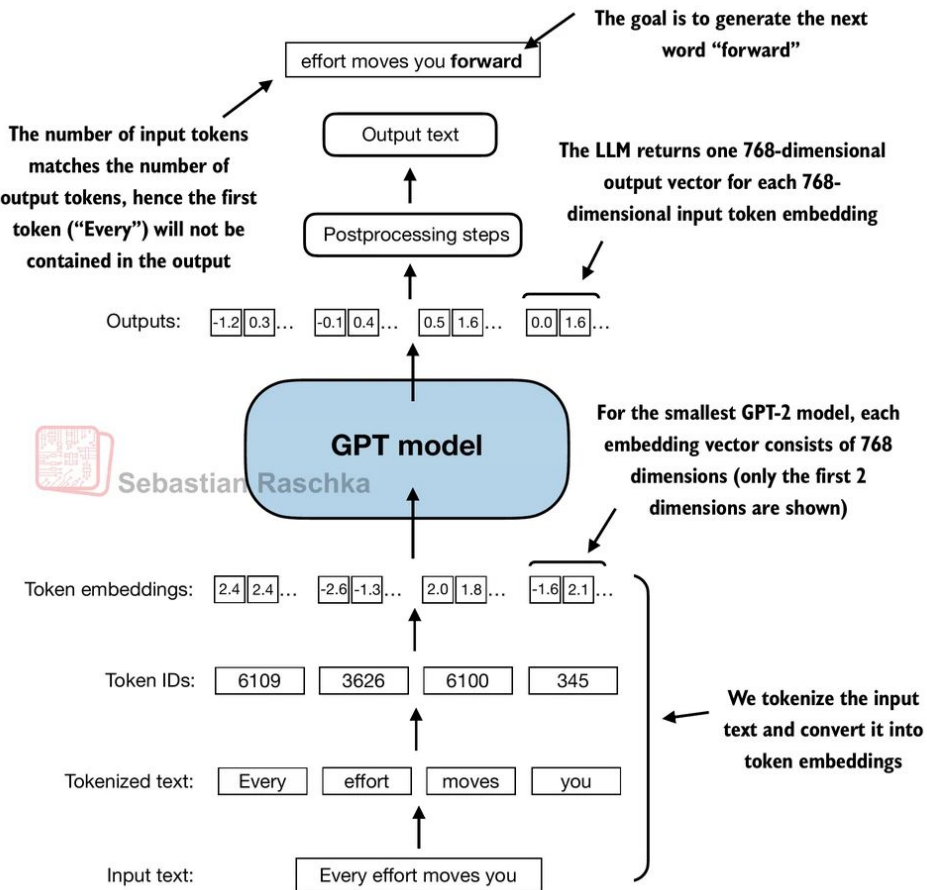
Contents



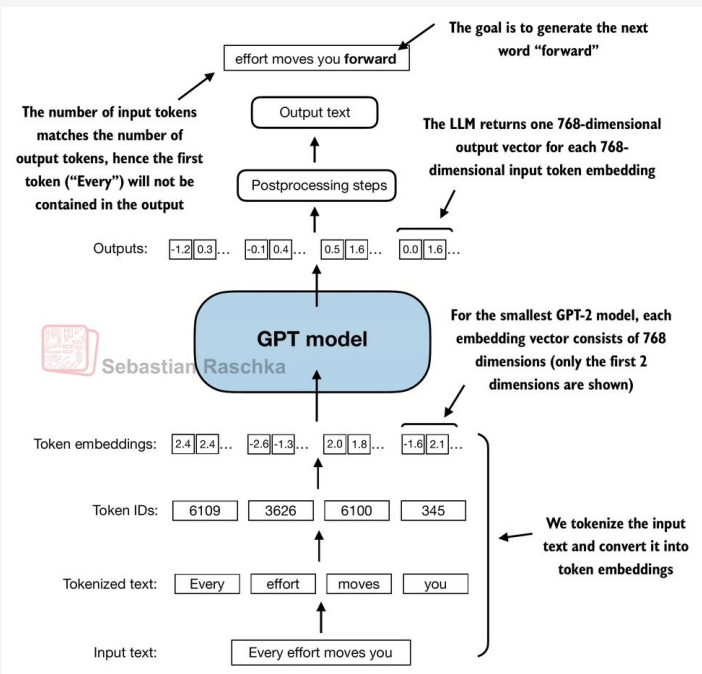
GPT backbone

Configuration

```
GPT_CONFIG_124M = {  
    "vocab_size": 50257, # Vocabulary size  
    "context_length": 1024, # Context length  
    "emb_dim": 768, # Embedding dimension  
    "n_heads": 12, # Number of attention heads  
    "n_layers": 12, # Number of layers  
    "drop_rate": 0.1, # Dropout rate  
    "qkv_bias": False # Query-Key-Value bias  
}
```



- "vocab_size" indicates a vocabulary size of 50,257 words.
- "context_length" represents the model's maximum input token count.
- "emb_dim" is the embedding size for token inputs, converting each input token into a 768-dimensional vector.
- "n_heads" is the number of attention heads in the multi-head attention mechanism.
- "n_layers" is the number of transformer blocks within the model, which we'll implement in upcoming sections
- "drop_rate" is the dropout mechanism's intensity, means dropping 10% of hidden units during training to mitigate overfitting.
- "qkv_bias" decides if the Linear layers in the multi-head attention mechanism (from Chapter 3) should include a bias vector when computing query (Q), key (K), and value (V) tensors.

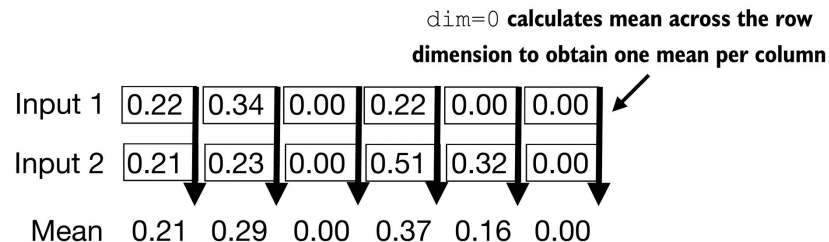
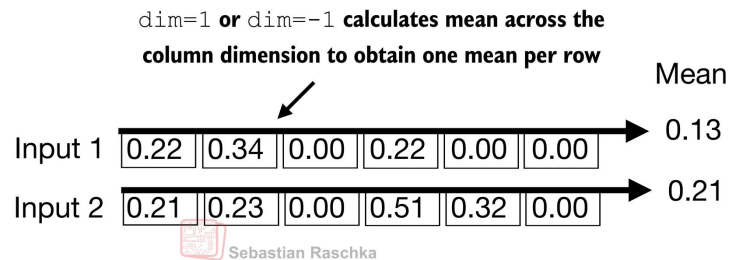
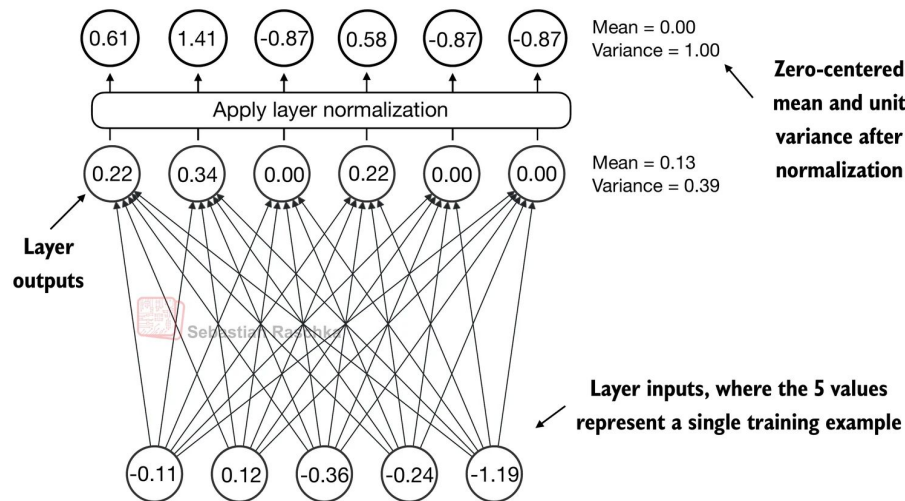


```

1 import torch
2 import torch.nn as nn
3
4
5 class DummyGPTModel(nn.Module):
6     def __init__(self, cfg):
7         super().__init__()
8         self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
9         self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
10        self.drop_emb = nn.Dropout(cfg["drop_rate"])
11
12        # Use a placeholder for TransformerBlock
13        self.trf_blocks = nn.Sequential(
14            *[DummyTransformerBlock(cfg) for _ in range(cfg["n_layers"])]
15        )
16
17        # Use a placeholder for LayerNorm
18        self.final_norm = DummyLayerNorm(cfg["emb_dim"])
19        self.out_head = nn.Linear(
20            cfg["emb_dim"], cfg["vocab_size"], bias=False
21        )
22
23    def forward(self, in_idx):
24        batch_size, seq_len = in_idx.shape
25        tok_embs = self.tok_emb(in_idx)
26        pos_embs = self.pos_emb(torch.arange(seq_len, device=in_idx.device))
27        x = tok_embs + pos_embs
28        x = self.drop_emb(x)
29        x = self.trf_blocks(x)
30        x = self.final_norm(x)
31        logits = self.out_head(x)
32        return logits
33
34 class DummyTransformerBlock(nn.Module):
35     def __init__(self, cfg):
36         super().__init__()
37         # A simple placeholder
38
39     def forward(self, x):
40         # This block does nothing and just returns its input.
41         return x
42
43
44 class DummyLayerNorm(nn.Module):
45     def __init__(self, normalized_shape, eps=1e-5):
46         super().__init__()
47         # The parameters here are just to mimic the LayerNorm interface.
48
49     def forward(self, x):
50         # This layer does nothing and just returns its input.
51         return x

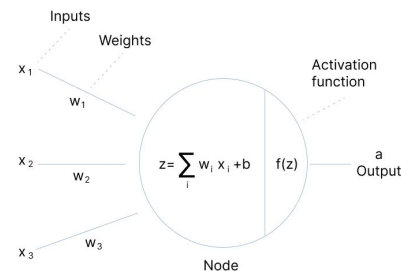
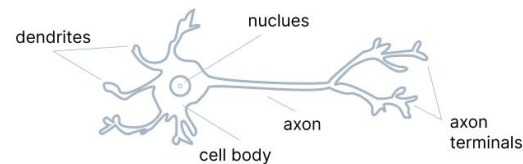
```

Layer normalization

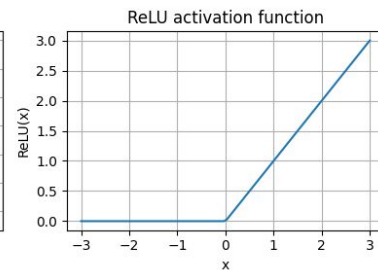
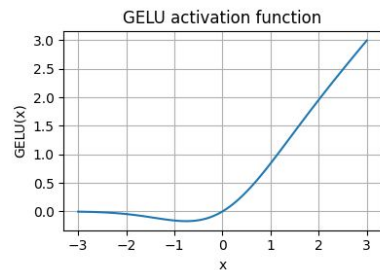


GELU (Gaussian Error Linear Unit) activation

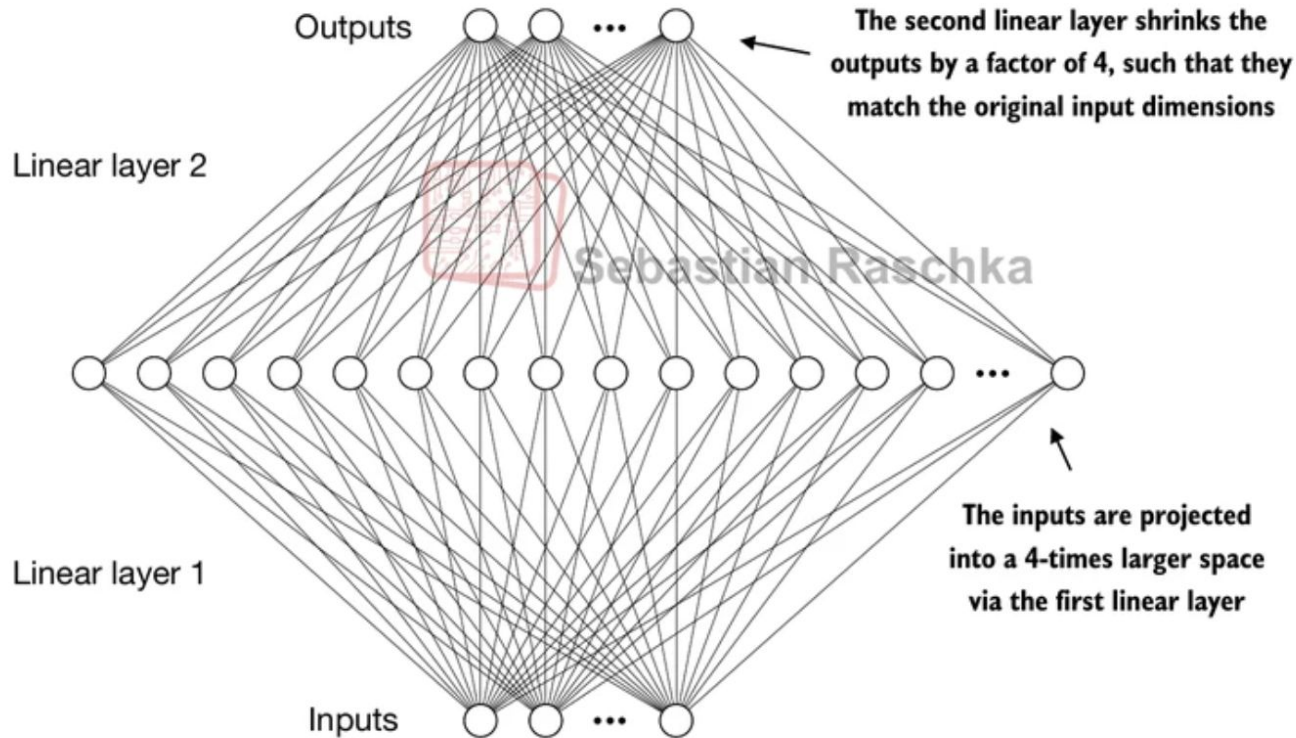
- **Definition:** Activation functions are mathematical equations that determine if a neuron should fire, introducing non-linearity to neural networks to enable the modeling of complex data.
- By mapping input signals to non-linear outputs, these functions (like ReLU, Sigmoid, or Tanh) allow networks to learn complex patterns beyond simple linear regressions.
- **How They Work:** They operate on a neuron's weighted sum of inputs, acting as a gate that decides if the information is relevant or should be ignored.
- **Key Properties:** Essential properties include being non-linear, continuously differentiable (for gradient-based learning), and having a range that helps manage data scaling.



V7 Labs



Feed forward network – “classic” NN architecture



Input: (2, 3, 3072)
Output: (2, 3, 768)

Input: (2, 3, 3072)
Output: (2, 3, 3072)

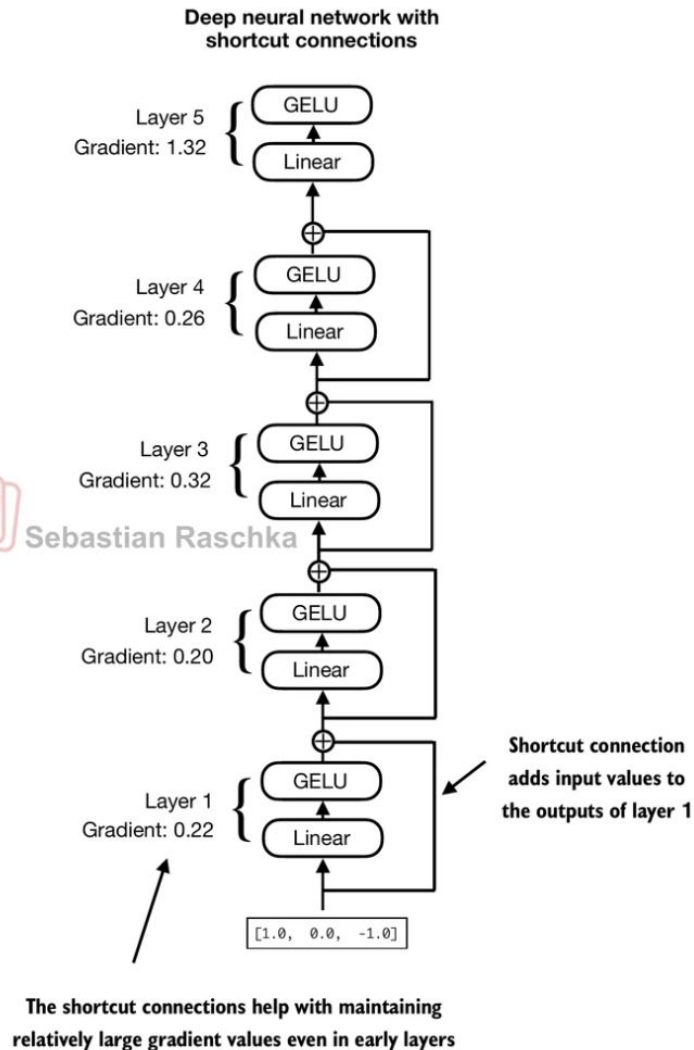
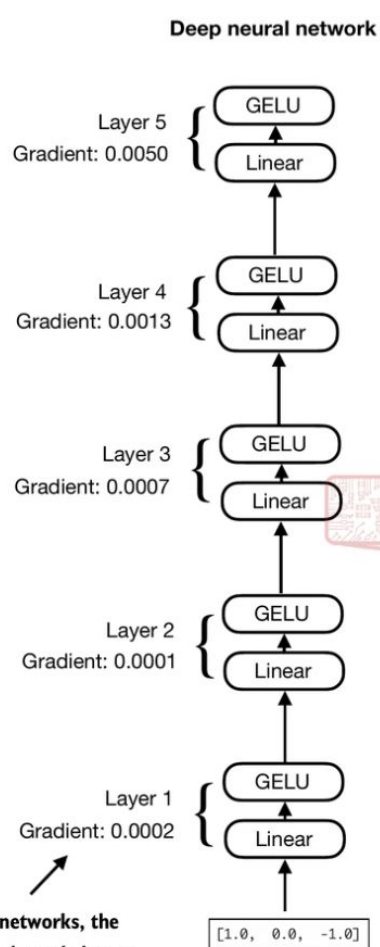
Input: (2, 3, 768)
Output: (2, 3, 3072)

Shortcut connections

- Prevent vanishing gradients
- Improved information flow

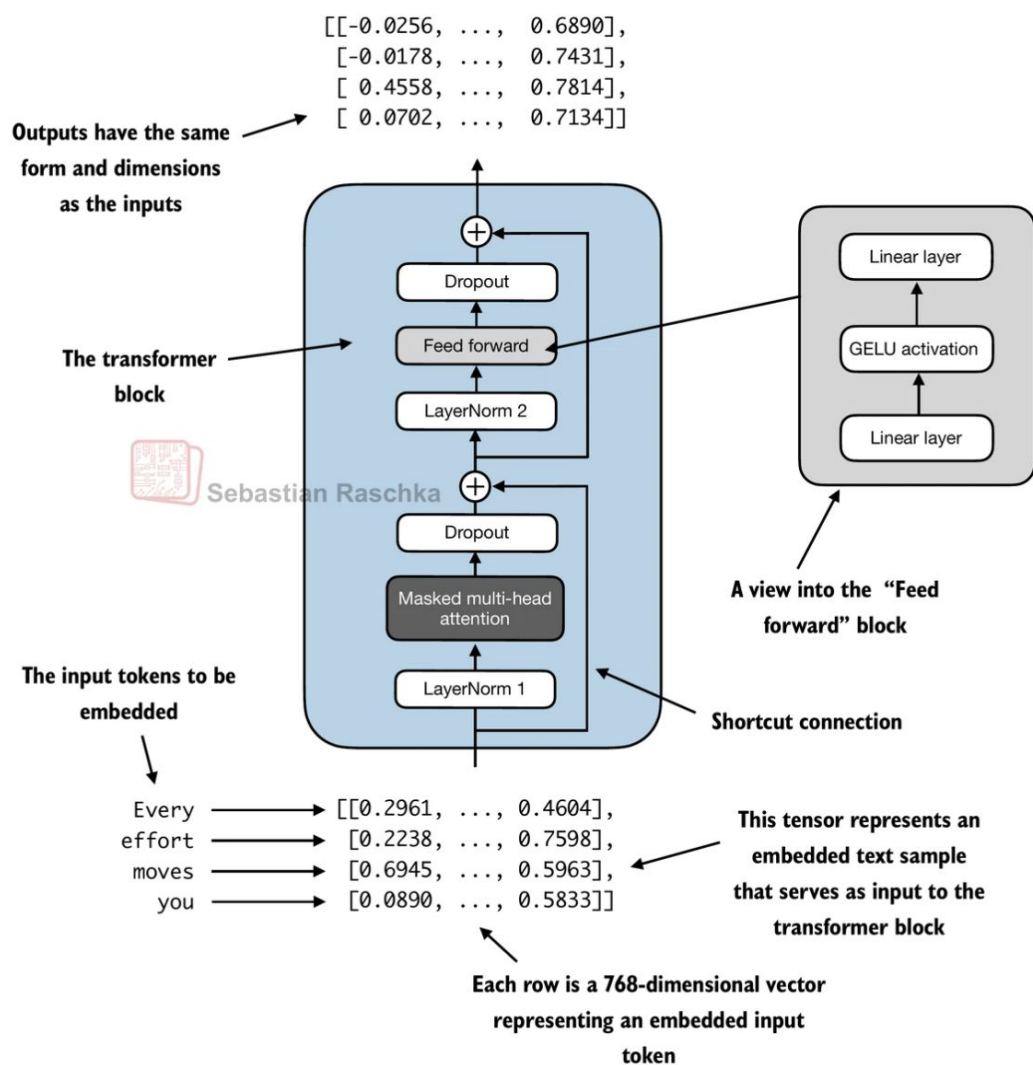
$$\frac{\partial H}{\partial x} = \frac{\partial F}{\partial x} + 1$$

In very deep networks, the gradient values in early layers become vanishingly small



```
1 class ExampleDeepNeuralNetwork(nn.Module):
2     def __init__(self, layer_sizes, use_shortcut):
3         super().__init__()
4         self.use_shortcut = use_shortcut
5         self.layers = nn.ModuleList([
6             nn.Sequential(nn.Linear(layer_sizes[0], layer_sizes[1]), GELU()),
7             nn.Sequential(nn.Linear(layer_sizes[1], layer_sizes[2]), GELU()),
8             nn.Sequential(nn.Linear(layer_sizes[2], layer_sizes[3]), GELU()),
9             nn.Sequential(nn.Linear(layer_sizes[3], layer_sizes[4]), GELU()),
10            nn.Sequential(nn.Linear(layer_sizes[4], layer_sizes[5]), GELU())
11        ])
12
13    def forward(self, x):
14        for layer in self.layers:
15            # Compute the output of the current layer
16            layer_output = layer(x)
17            # Check if shortcut can be applied
18            if self.use_shortcut and x.shape == layer_output.shape:
19                x = x + layer_output
20            else:
21                x = layer_output
22        return x
23
24
25 def print_gradients(model, ...):
26     # Forward pass
27     output = model(x)
28     target = torch.tensor([0.])
29
30     # Calculate loss based on how close the target
31     # and output are
32     loss = nn.MSELoss()
33     loss = loss(output, target)
34
35     # Backward pass to calculate the gradients
36     loss.backward()
37
38     for name, param in model.named_parameters():
39         if 'weight' in name:
40             # Print the mean absolute gradient of the weights
41             print(f"{name} has gradient mean of {param.grad.abs().mean().item()}")
```

Transformer block



2) The GPT model returns a matrix consisting of 4 vectors (rows), where each vector has 50257 dimensions (columns)

1) Encode text input into 4 token IDs

"Hello"
", "
"I"
"am"

"a"

[15496,
11,
314,
716]

257

GPT

$\begin{bmatrix} [-0.2949, \dots, -0.8141], \\ [1.2199, \dots, -0.3599], \\ [1.0446, \dots, 0.0020], \\ [-0.4929, \dots, -0.6093] \end{bmatrix}$

3) Extract the last vector, which corresponds to the next token that the GPT model is supposed to generate

Sebastian Raschka

Logits: $[-0.4929, \dots, 2.4812, \dots, -0.6093]$

Softmax

Probabilities: $[0.0001, \dots, 0.0200, \dots, 0.0001]$

4) Convert logits into probability distribution using the softmax function

5) Identify the index position of the largest value, which also represents the token ID

0

257

50257

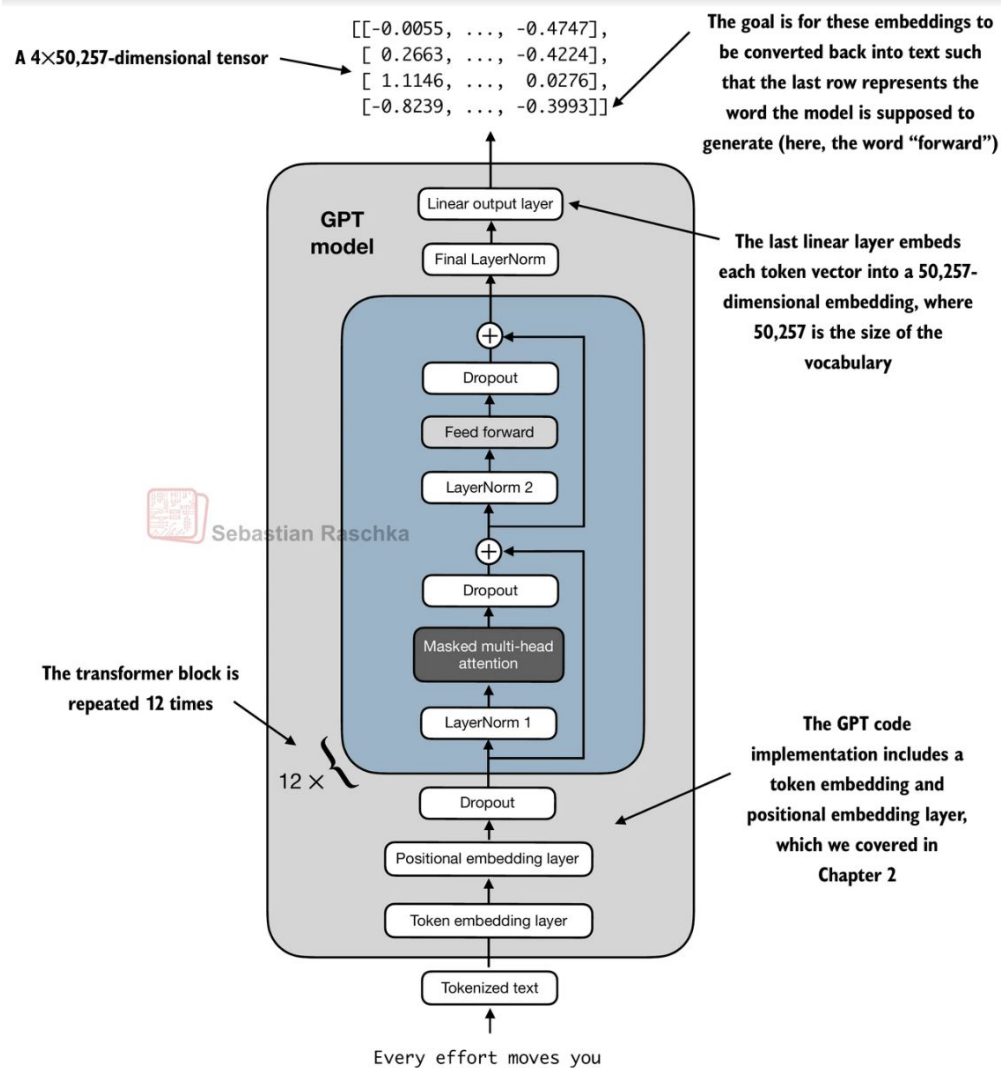
6) Append token to the previous inputs for the next round

Token ID decoded into text

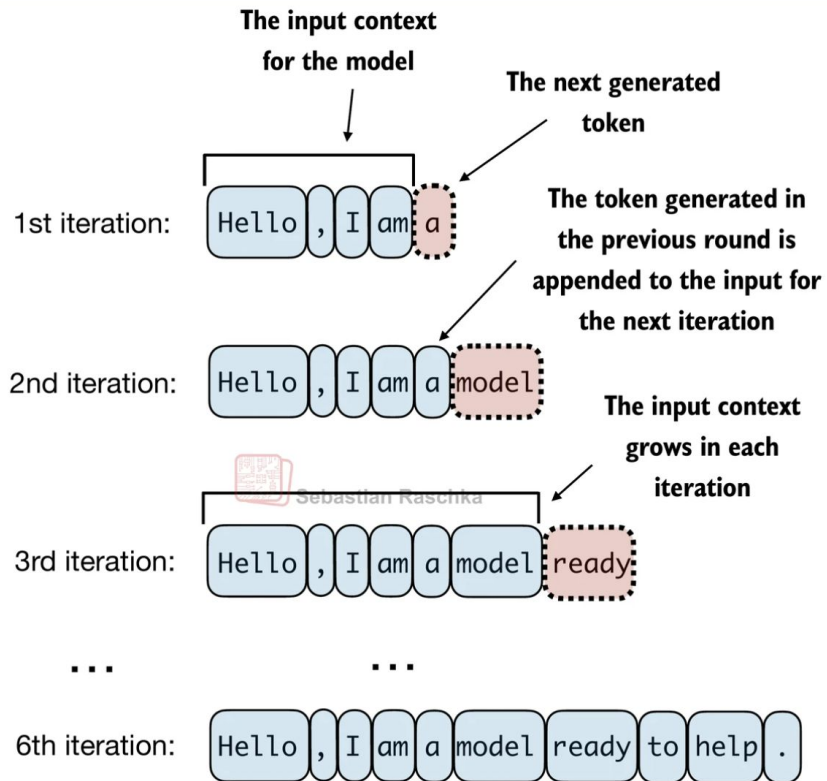
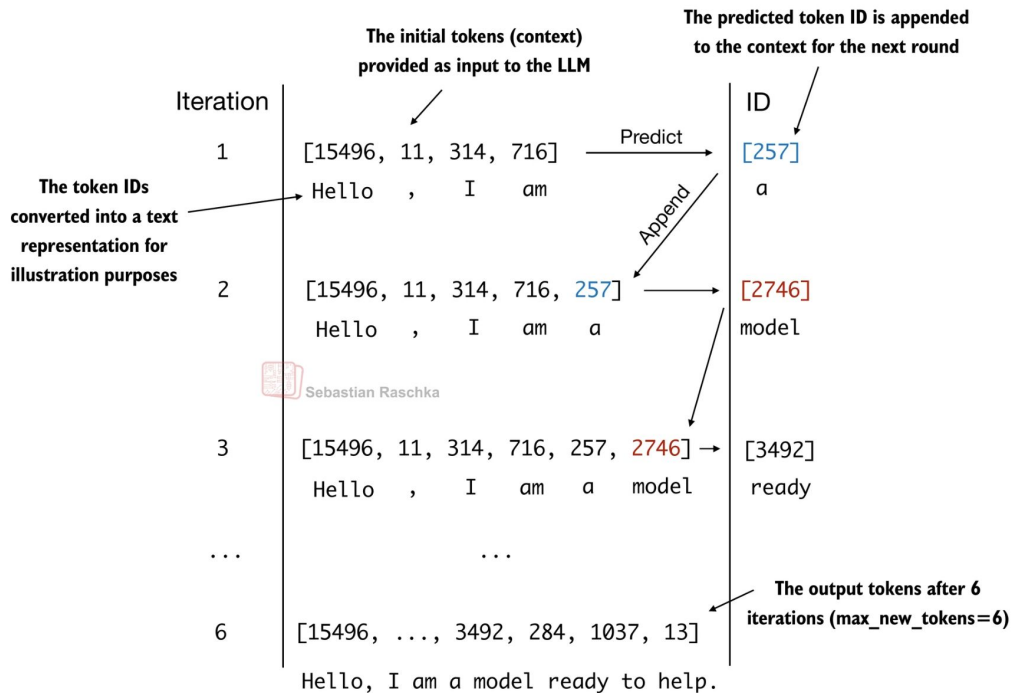
"a"

If the largest element is at position 257, we obtain token ID 257

GPT-2



Autoregressive Generation



Summary

